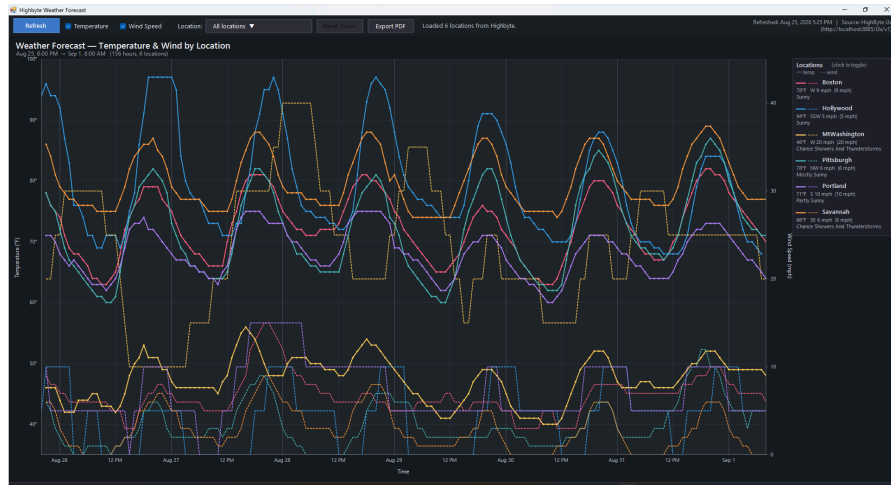


WeatherPlot

A standalone Windows desktop app that plots hourly weather forecasts (temperature + wind speed) for multiple locations on a single interactive chart. Data is sourced from a [HighByte Intelligence Hub](#) via the [i3x industrial data API](#), with the National Weather Service's hourly forecast underneath.



1. Quick Start

1. Double-click `WeatherPlot.exe`.
2. The **Connect to HighByte i3x** dialog appears (see §1.1). Enter the server URL (host root, no path) and either your credentials or a pre-issued bearer token, then click **Connect**.
3. On success, the main chart window opens. It loads from `weather_data.json` (if present) immediately, then refreshes in the background by calling the i3x API. New cache is written on every successful refresh, so the next startup works even offline (once you've authenticated).

No installer, no .NET SDK, no NuGet packages — just a single `.exe` plus a small JSON cache file and a tiny `connection.json` beside it.

1.1 The Connect Dialog

When the app launches, a modal **Connect to HighByte i3x** dialog comes up first. The main window never appears until you successfully authenticate (or cancel out, which exits the app).

The dialog has:

- **Server URL** — defaults to `http://localhost:8885`. This is the **host root**, not the i3x path. The app appends `/i3x/v1/...` to every data call. If you omit the scheme, `http://` is added automatically; trailing slashes are stripped.
- A mode toggle:
- **Bearer Token (default)** — paste a pre-issued token directly. Every subsequent request carries `Authorization: Bearer <token>`. This is the standard path for i3x deployments, since the i3x standard does not itself define a login endpoint; tokens are typically provisioned server-side.
- **No Auth** — send no authorization at all. The `Authorization` header is omitted from every request. Use this for open i3x servers (test/dev instances, servers running on a trusted local network without access controls). If the server actually does require auth, the first data call will surface an HTTP 401 in the toolbar status bar.
- **Connect / Cancel** buttons. **Enter** triggers Connect; **Esc** triggers Cancel.

Neither mode performs a server round-trip during the Connect step itself — the settings are captured and MainForm opens immediately. Any auth problems surface on the first data refresh (401 for a bad/missing token) and appear in the toolbar status bar.

Persistence

The dialog reads and writes a small `connection.json` file next to the `.exe`:

```
{
  "Url": "http://localhost:8885",
  "Mode": "token",
  "TokenProtected": "AQAAANCMnd8BFdERjHoAwE/Cl+sBAAAA...(base64)..."
}
```

Mode is "token" (Bearer Token) or "noauth" (No Auth).

The bearer token is persisted encrypted at rest via Windows DPAPI (ProtectedData.Protect with DataProtectionScope.CurrentUser). What this means in practice:

- On next launch the token field is pre-populated so you don't have to paste it every session.
- The value on disk is a base64-encoded encrypted blob, not the plaintext token. Casual inspection of connection.json reveals nothing.
- Decryption only succeeds for **the same Windows user on the same machine**. Copying connection.json to another PC, or opening it under a different user account, yields an empty token — you'll be asked to paste it again.
- If the token needs to change (rotation, moving to a different server), just paste the new value and click **Connect**. The persisted blob is refreshed on every successful Connect, so the new value replaces the old one.

To wipe the persisted token, delete connection.json (or just the TokenProtected line) and re-launch the app.

2. The Chart

The chart fills the main window. It plots **time** on the X-axis and supports two simultaneous Y-axes:

Axis	Side	Series style	Units
Temperature	Left	Solid line, 2 px	°F
Wind Speed	Right	Dashed line, 1.6 px	mph

Each location is rendered in its own color (from a fixed 8-color palette). Both temperature and wind-speed lines for one location share that color, so you read by hue and distinguish series type by line style.

Chart elements

- **Title**: reflects what series are shown ("Temperature & Wind by Location", "Temperature by Location", "Wind Speed by Location", or "(no series selected)").
- **Subtitle**: shows the visible time range, count of hours, count of visible locations, and a [zoomed: X of Y h] indicator when zoomed.
- **Gridlines**: horizontal at "nice" temperature steps (e.g. every 5°F); vertical at hourly markers (every 2/6/12 hours depending on zoom). Day boundaries get a slightly brighter vertical line.
- **Hover tooltip**: when the cursor is near a data point, a tooltip appears showing the location name, day/time, temperature in °F, wind direction + speed, and the NWS short forecast (e.g. "Rain Showers Likely"). Tooltip indicates whether you've snapped to a (temp) or (wind) point.
- **Legend panel** (right side): one card per location showing the line-style legend (— temp, - wind), name, current temperature, current wind direction + speed, and current forecast. Hidden locations are dimmed with a strike-through on the name.
- **Empty state**: if no data is loaded yet, the chart says "No data loaded. Click Refresh to fetch current forecast." If all locations or all series types are turned off, an informative subtitle appears in place of the time range.

3. Toolbar

The top toolbar (44 px tall, left to right):

Control	Behavior	
Refresh	Pulls fresh data from i3x: <code>GET /i3x/v1/objects</code> (filter children of <code>Weather</code>) then <code>POST /i3x/v1/objects/value</code> per location with <code>{"elementIds":["Weather.<name>.Forecast"]}</code> . Updates the chart and overwrites <code>weather_data.json</code> .	
Temperature checkbox	Show/hide the solid temperature lines and the left Y-axis.	
Wind Speed checkbox	Show/hide the dashed wind-speed lines and the right Y-axis.	
Location: dropdown button	Opens a popup with Select All / Unselect All + a checkbox list of every location (see §4). The button label summarizes the current selection.	
Reset Zoom	Restore the full data time range. Dimmed when not zoomed; brightens when a zoom is active.	
Export PDF	Capture the current chart view (whatever's currently visible — selected locations, zoom level, axis visibility, hover tooltip excluded) and save it as a PDF. See §5.1.	
Status label	Live progress text during refresh (<code>"Fetching Pittsburgh forecast ... (4/6)"</code>) or summary after load (<code>"Loaded 5 locations from Highbyte."</code>). Shows errors if any locations failed.	
Refreshed / Cached label (far right)	Last refresh time + source URL, e.g. `Refreshed: May 24, 2026 9:28 PM`	Source: Highbyte (<code>http://localhost:8885</code>) . If a refresh hasn't run yet, shows the cache timestamp.

4. Multi-Select Location Dropdown

Click the **Location:** button to open the dropdown popup. It contains:

- A bulk-action row at the top with two buttons:
- **Select All** (blue) — checks every location at once.
- **Unselect All** (gray) — clears every checkbox at once (the chart goes empty, showing the *"No locations visible"* state).
- A checkbox list of every location returned by Highbyte. Click any row to toggle that location's visibility.

The chart updates **live** as you check / uncheck — no need to close the popup. Click anywhere outside the popup to dismiss it. Both bulk-action buttons leave the popup open so you can adjust further (e.g. *Unselect All* → manually check just the two locations you want).

Button label conventions

Selection state	Button text
Everything checked (default)	All locations ▼
Nothing checked	(none selected) ▼
Exactly 1 checked	<Name> ▼
Exactly 2 checked	<Name1>, <Name2> ▼
3+ checked	N locations selected ▼

Persistence

When you refresh data, your selection is preserved by location name. New locations added on the Highbyte side default to checked. Removed locations silently disappear.

The location dropdown and the legend-row click toggle are two independent ways to do the same thing — they stay in sync.

5. Chart Interactions

Click

- Click a **legend row** → toggle that location's visibility (chart rescales axes to fit what's left). Equivalent to checking/unchecking that row in the dropdown.

Hover

- **Move the cursor over the chart** → snap-to-nearest-point tooltip appears. Both temperature and wind points are candidates.
- **Hover a legend row** → cursor turns into a pointing hand to indicate clickability.

Scroll wheel — Zoom

- **Scroll up** over the chart → zoom in on the time axis by 1/1.25× per notch.
- **Scroll down** → zoom out by 1.25× per notch.
- Zoom is **centered at the cursor's X position**, so the time you're pointing at stays anchored as you zoom.
- The Y-axis (and wind axis) auto-rescale to fit the data within the visible window, so a narrow zoom over a single day shows that day's actual temperature range using the full chart canvas.
- Hard limits: can't zoom in tighter than 1 hour; can't zoom out beyond the full data range.
- The subtitle gains a `[zoomed: 40 of 155 h]` annotation when active.
- Click **Reset Zoom** in the toolbar to restore the full range.

Zoom level is preserved across location toggles, checkbox toggles, and data refreshes (clamped to the new data range if needed).

5.1 Export PDF

Click the **Export PDF** button in the toolbar to save the current chart view as a PDF document.

- A standard *Save File* dialog appears. The default filename is `WeatherPlot_<YYYYMMDD_HHMM>.pdf` ; pick any name and location.
- The PDF is rendered in **landscape Letter** size with 40 px margins. The chart is scaled to fit the page while preserving its aspect ratio, centered with letterboxing if necessary.
- The exported chart matches exactly what's on screen: which locations are visible, which series types (temp / wind) are on, the current zoom range, and the auto-rescaled axes for that zoom. The interactive hover tooltip is intentionally **not** included.
- Implementation: `ChartPanel.RenderToBitmap()` re-runs the chart's `OnPaint` against an in-memory `Graphics` , and a `PrintDocument` configured for **Microsoft Print to PDF** writes the bitmap to the chosen file path. No external PDF library or NuGet package is involved — this is all built into the .NET Framework on Windows.
- If the *Microsoft Print to PDF* printer isn't installed on the machine, you'll see a clear error message and a hint to enable it under *Settings* → *Printers & scanners*.

6. Data Source

i3x endpoints

The app calls three endpoints on the HighByte Intelligence Hub, one login helper plus two i3x-standard reads:

Endpoint	HTTP	Body / Notes	Returns
Discover locations	GET /i3x/v1/objects	Standard i3x catalog endpoint. Returns every element in the hierarchy. Client-side filters to <code>parentId == "Weather"</code> and takes each entry's <code>displayName</code> .	<code>{"success":true, "result":[{"elementId":"Weather.Boston","displayName":"Boston", "...}, ...]}</code>
Read forecast	POST /i3x/v1/objects/value	Standard i3x current-value endpoint. Body: <code>{"elementIds":["Weather.<name>.Forecast"]}</code> — one call per location.	<code>{"success":true, "results":[{"elementId":"Weather.Boston.Forecast","result":{"valu [{"Time":"...", "Temperature":78, "WindSpeed":"9 mph", "Forecast":"Sunny"}, ...]}}</code>

The `Periods` array is the sequence of hourly forecast points. Each point has `Time` (ISO-8601 with offset), `Temperature` (integer °F), `WindSpeed` (string like "12 mph"), `WindDirection`, and `Forecast` (NWS short text).

Data model

The i3x hierarchy this app assumes:

Weather

- Boston
 - Position
 - Current
 - Forecast ← type:Forecast
- Hollywood / same structure
- MtWashington / same structure
- Pittsburgh / same structure
- Portland / same structure
- Savannah / same structure

(root object)
(location – displayName is what appears in the UI)
(unused by this app)
(unused by this app)
(this is what the app reads)

Locations are discovered dynamically from the server — the app doesn't hard-code names. New locations added under `Weather` on the server appear automatically after the next Refresh.

Authentication

- Bearer Token mode:** every i3x request carries `Authorization: Bearer <token>` with the token pasted into the Connect dialog. Tokens must be provisioned server-side (see the HighByte Intelligence Hub admin UI).
- No Auth mode:** the `Authorization` header is omitted entirely. Suitable for open i3x servers on trusted networks; if the server actually requires auth you'll see HTTP 401 in the toolbar status bar on the first refresh.

TLS 1.2 is explicitly enabled for HTTPS deployments in either mode.

Cache (`weather_data.json`)

Lives next to the .exe. Format:

```

{
  "Locations": [
    {
      "Name": "Boston",
      "Forecast": [
        { "Time": "...", "Temperature": 52, "WindSpeed": "12 mph",
          "WindDirection": "E", "Forecast": "Rain Showers" },
        ...
      ]
    },
    ...
  ]
}
```

The cache is loaded synchronously on startup (instant display), then overwritten on every successful refresh.

7. Configuration

All connection configuration is done **interactively at startup** via the Connect dialog (see §1.1). There are no environment variables, command-line flags, or config files for URL/credentials.

The dialog persists three settings to `connection.json` next to the .exe so the next launch is one-click for the same server:

Field	Persisted?	Purpose
Url	yes	Pre-fills the Server URL field
Mode	yes	Selects which radio button is active when the dialog opens ("token" or "noauth")
TokenProtected	yes, DPAPI-encrypted per user/machine	Pre-fills the Bearer Token field. Base64 of a <code>ProtectedData.Protect(..., DataProtectionScope.CurrentUser)</code> blob — never plaintext on disk. Refreshed on every successful Connect.

If `connection.json` is missing or corrupt, defaults (`http://localhost:8885` , Bearer Token mode) are used.

8. Files

```
C:\Code\Claude\WeatherPlot\  
├─ WeatherPlot.exe          - the program  
├─ WeatherPlot.cs           - single-file C# source  
├─ weather_data.json        - local cache, auto-managed  
├─ connection.json          - saved URL + username + mode (no password/token)  
├─ screenshot*.png          - verification screenshots  
├─ README.md                - this file  
└─ README.pdf               - printable version of this file
```

Single source file, single binary. No DLLs, no installer. `connection.json` is created on first successful login.

9. Building from Source

The project compiles with the .NET Framework 4.x C# compiler that ships with Windows — **no .NET SDK required**.

```
$csc = "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe"  
& $csc /nologo /target:winexe /platform:x64 /out:WeatherPlot.exe `   
  /reference:System.dll `   
  /reference:System.Core.dll `   
  /reference:System.Drawing.dll `   
  /reference:System.Windows.Forms.dll `   
  /reference:System.Web.Extensions.dll `   
  WeatherPlot.cs
```

The only references are framework assemblies (`System.Web.Extensions` is used for its `JavaScriptSerializer` , since `System.Text.Json` isn't available in .NET Framework 4.x).

Target runtime: any Windows 7+ machine with .NET Framework 4.x — already present on every modern Windows install.

10. Architecture



Key classes

Class	Role
LoginForm	Modal Connect dialog: server URL + auth, calls <code>/data/v1/login</code> , configures <code>HighbyteClient</code>
ConnectionSettings	Load/save <code>connection.json</code> (URL + username + mode, never secrets)
MainForm	Form, toolbar wiring, refresh coordination, cache I/O
ChartPanel	Custom-drawn chart (GDI+), zoom/hover/legend interaction
I3xClient	HTTP client for login helper + the two i3x-standard reads (<code>GET /objects</code> , <code>POST /objects/value</code>); configured at runtime via <code>Configure(url, token)</code>
WindParser	Extract leading numeric value from strings like <code>"12 mph"</code>
LocationSeries	Per-location data: <code>Times[]</code> , <code>Temperatures[]</code> , <code>WindSpeeds[]</code> , <code>Visible</code> , <code>Color</code>
ForecastPoint	One forecast hour: <code>Time</code> , <code>Temperature</code> , <code>WindSpeed</code> , <code>WindDirection</code> , <code>Forecast</code>

Threading

- HTTP fetches run on a thread-pool thread (`ThreadPool.QueueUserWorkItem` in `BeginRefresh`).
- UI updates marshal back via `BeginInvoke` (helper: `MainForm.UiInvoke`).
- During refresh, the **Refresh** button is disabled, the status label streams progress, and the chart only updates once at the end.

11. Defensive Design Notes

This app picked up a few hardenings during development:

- **OnLoad state reset + formReady flag**: handlers no-op while the form is initializing, then `OnLoad` explicitly asserts the known-good defaults. Prevents spurious events (e.g. from injected keystrokes during window show) from corrupting initial state.
- **MouseClicked instead of Click on the toolbar checkboxes**: spacebar / synthetic keys can fire `Click` but not `MouseClicked`. `AutoCheck = false` removes the built-in toggle so only our mouse-driven handler can change the checked state.

- `SelectionChangeCommitted` instead of `SelectedIndexChanged` on the historical `ComboBox` (now replaced by the multi-select): the committed event fires only on user-confirmed selections, not on programmatic changes or first-letter auto-jumps.
- `TabStop = false` on the checkboxes, dropdown button, chart panel, etc., so Tab navigation doesn't park focus anywhere accidentally toggleable. `ActiveControl = refreshBtn` parks initial focus on the safest control.
- **Keyboard-event suppression** on interactive controls where it matters.
- `suppressLocChange` guard during `RebuildLocationDropdown` to prevent re-entrant `ItemCheck` events from spawning duplicate chart rerenders.
- **Z-order-aware docking**: Fill children added before Top/Right/Bottom children so WinForms' reverse-order docking layout gives Fill the remaining space.

12. Keyboard / Mouse Reference

Action	Result
Click Refresh	Re-fetch all locations from Highbyte
Click Temperature / Wind Speed	Toggle that series type globally
Click Location: ▼	Open multi-select dropdown
Click Select All in popup	Check every location
Click Unselect All in popup	Clear every checkbox
Click a row in popup	Toggle that location
Click outside popup	Close popup
Click a legend row on the chart	Toggle that location (alt path)
Move cursor over the chart	Show snap-to-nearest tooltip
Scroll up over the chart	Zoom in at cursor
Scroll down over the chart	Zoom out at cursor
Click Reset Zoom	Restore full time range
Click Export PDF	Save the current chart view as a PDF (Save dialog, then writes via Microsoft Print to PDF)

13. Known Limitations / Future Ideas

- The default 6-location palette has 8 colors, so up to 8 locations get distinct colors before it cycles.
- Wind speed parsing handles `"12 mph"` and `"10 to 15 mph"` (uses the leading number). Other formats from upstream data sources may not parse perfectly.
- No drag-to-pan once zoomed (you can zoom out and re-zoom on a different region, or use scroll-down to zoom out incrementally).
- No keyboard shortcuts (e.g., R to refresh, Z to reset zoom). Easy to add if requested.
- No persistent user preferences (visibility, zoom) across launches. Session-only.
- No HTTPS certificate validation override — uses system defaults (TLS 1.2 explicitly enabled).

14. Version History

- **v1** — Initial WinForms chart, JSON cache, NWS direct fetch
- **v2** — Switched to Highbyte pipelines for data
- **v3** — Wind speed dual-axis, click-to-toggle legend
- **v4** — Single-select location dropdown
- **v5** — Multi-select location dropdown (CheckedListBox popup)
- **v6** — Scroll-wheel zoom + Reset Zoom button
- **v7** — Select All button in dropdown popup
- **v8** — Unselect All button beside Select All in dropdown popup

- **v9** — Connect dialog at startup (Username/Password → `/data/v1/login` for bearer token, or paste-a-token mode); `connection.json` persists URL + username + mode
- **v10** — Export PDF button in toolbar (renders the current chart view to a PDF via `PrintDocument` + *Microsoft Print to PDF*, landscape Letter, aspect-preserved)
- **v11** — Switched data source from HighByte proprietary REST pipelines to the vendor-neutral [i3x industrial API](#). Locations are now discovered dynamically as children of the `Weather` root object; per-location forecasts come from `POST /i3x/v1/objects/value`. Dialog and status labels updated. `HighbyteClient` renamed `I3xClient`.
- **v12** — Removed the Username & Password mode from the Connect dialog; replaced with a **No Auth** mode that omits the `Authorization` header entirely for open i3x servers. `connection.json` schema simplified (`Mode` string replaces `Username` + `UseToken`).
- **v13** — Bearer token is now persisted across launches, encrypted at rest with Windows DPAPI (`ProtectedData / DataProtectionScope.CurrentUser`). The token field pre-populates on next launch; the persisted blob is refreshed on every successful Connect so a changed value updates automatically.